BRAVO PRODUCTION FINDINGS · 04 OF 05 · COST

Where Bravo's money actually goes

> *Three questions. What does Bravo cost, compared fairly against LORA? What drives that cost? And which savings are real — how much is each one worth, and when can we take it?*

Rp 58M THE PART OF BRAVO THAT RUNS LOAN WORKFLOWS. THE SAME PART OF LORA COSTS RP 431M.	**Rp 140.5M** CLOUD LOGGING ON ITS OWN. THAT IS 2.4 TIMES THE WHOLE WORKFLOW TIER.	**Rp 573.5M** TEST AND STAGING ENVIRONMENTS. 31% OF ALL BRAVO SPEND, AND NOBODY HAS LOOKED AT IT.	**4.5%** THE WORKFLOW ENGINE'S SHARE OF BRAVO'S OWN PRODUCTION BILL.

Audience Platform, finance, and the CTO office sizing the migration **Sources** GCP billing via `bfi-finops-dashboard` · complete month of August 2026 **Read** 2026-09-10

Six claims, checked against a fresh pull of the bill

Bravo is expensive	● NOT THE PART THAT RUNS WORKFLOWS	**Bravo is the cheaper platform here.** The `ms-bpm` pods plus their database cost **about Rp58M a month in production**. The same layer in LORA costs **about Rp431M**. That is roughly **7 times cheaper** in total, and **5 to 6.5 times cheaper per application**. We pulled the bill again independently, and it confirms the earlier finding.
Retiring Bravo saves about Rp1.6B a month	● WITHDRAWN	We withdrew this claim before, and the new pull confirms the withdrawal. The Bravo GCP projects do bill **Rp1.874B** in August. But almost everything beyond the workflow tier is the **shared BFI data plane** — about 26 `ms-*` services that *LORA itself calls*. Those do not go away. What the migration actually retires is **about Rp58M in production, or Rp70–100M including test environments**. That is 8 to 14 times the cost of the whole Temporal Actions programme, not 227 times.
The workflow engine is what makes Bravo expensive	● NO	The workflow tier is **4.5% of Bravo's own production project**. Cloud SQL (Rp450.7M) and Compute Engine (Rp394.4M) make up 65% of `bravo-project-331802`, and most of neither one is the workflow engine.
Bravo gets cheaper per application as it drains	● REFUTED	Volume was **flat**: 117,996 applications in July, 118,253 in August. Support tickets **rose 82%** over eight months. The platform costs the same each month whatever the volume, so the cost per application is **going up**. Architecture has nothing to do with it.
The cost work worth doing is on the engine	● REFUTED — THREE BIGGER SAVINGS	Cloud Logging costs **Rp140.5M a month**, which is 2.4 times the whole workflow tier. Behind it sit one global setting, `loggerLevel: full`, and an error stream that is 47% stack-trace lines. The Maps APIs cost **Rp64.0M a month**. Test and staging cost **Rp573.5M**.
Bravo's cost is a reason to keep it	● PARTLY — AND IT IS THE STRONGEST ARGUMENT ON ITS SIDE	But the part that would retire is small, about Rp58M. And the platform it would move to costs more per application today. **The cost case for migrating is that LORA's cost for each extra application is close to zero, and that we would run one platform instead of two.** It is not that Bravo is expensive.

Bravo's workflow layer is genuinely cheap. And almost none of Bravo's bill is the workflow layer.

The money is in a database, a logging pipeline, and a set of test environments. Two of those three are misconfiguration, not real work.

Nobody sends us a bill for running Bravo's workflows

There is no count of workflow actions, no storage line per workflow, nothing like the Temporal Cloud invoice. Camunda 7 is a **library running inside a Spring Boot service**. So every cost it creates lands somewhere else on the GCP bill.

WHERE THE ENGINE'S COST SHOWS UP	BILL LINE	NOTES
Workflow state, history, and all 238 business entities	**Cloud SQL** — `prod-postgres-bpm-d2bpm`	History is set to `full` and kept for 90 days (`historyTimeToLive: P90D`). The `act_hi_actinst` table holds about 50 million rows in each 90-day window.
The threads that run all 483 service tasks	**Compute Engine** — `ms-bpm` pods on `prod-bravo-cluster`	The job executor is still on the Spring Boot default: a core pool of 3.
Every log line the engine and its controllers write	**Cloud Logging**	1,116,135 lines a week from `prod-ms-bpm` alone
Calls out to 25 other services	those services' own bills	—

WHAT THIS LETS US COMPARE, AND THE TRAP IN IT

Because the engine is a library inside the service, you cannot separate what Bravo's orchestration costs from what Bravo's human-task application costs. **They are the same deployment.** `ms-bpm` holds the BPMN engine *and* about 217,000 lines of surveyor, operation, underwriting and approval code — 44% of the codebase. So the Rp58M figure covers **orchestration plus all human-task handling**. That is the right comparison against LORA's Rp431M, which covers workers, gateway, task service, schema service, Temporal and ArangoDB. **Both sides include the human-task layer. That is what makes the ratio fair.**

August 2026, measured

PROJECT	AUGUST 2026
`bravo-project-331802` — production	**Rp 1,300,270,632**
`bravo-project-nonprod`	**Rp 573,504,402**
`bravo-project-shared-services`	Rp 13,568,891
`bravo-project-collection`	Rp 181,361
Bravo total	**Rp 1,887,525,286**
BFI GCP estate, all 34 projects	*Rp 3,708,544,980*

BRAVO IS 50.9% OF BFI'S ENTIRE GCP BILL

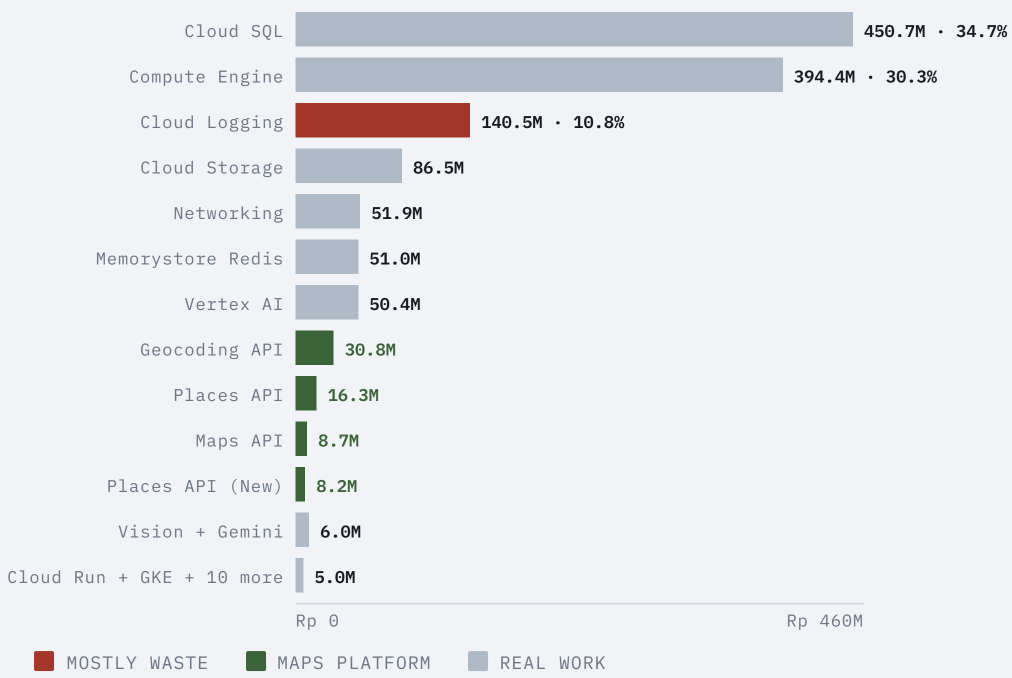
That number is striking, and it is the one most likely to be misread. So here it is precisely. It is the bill for the platform that hosts most of BFI's loan origination. It also covers the roughly 26 shared `ms-*` data services that **both platforms** depend on. **It is not the price of the workflow engine.**

TWO GROUPS WORTH NAMING, BECAUSE NEITHER APPEARS AS A SINGLE BILL LINE

Google Maps platform — Rp 63,979,414 a month (Geocoding, Places, Maps and Places New added together). That is more than the entire workflow tier.

AI and OCR — Rp 56,357,315 a month (Vertex AI, Vision and Gemini).

SERVICE BREAKDOWN, PRODUCTION PROJECT



What the workflow tier costs: about Rp58M

COMPONENT	AUGUST 2026, PRODUCTION	SOURCE
`prod-postgres-bpm-d2bpm` — CPU and memory	**Rp 40,930,777**	measured today, from 30 daily line items
… storage, backup and the rest	≈ Rp 11,800,000	the remainder of the instance total
Cloud SQL instance, total	**≈ Rp 52,700,000**	**cross-checked**
`ms-bpm` pods, production	≈ Rp 5,300,000	compare-architecture.md §8.4
Production workflow tier	**≈ Rp 58,000,000**	**—**
`ms-bpm` pods, SIT and UAT	≈ Rp 12,100,000	—
With test and staging pods	**≈ Rp 70,100,000**	**We did not pull the SIT, UAT and Sharia *databases* separately, so the real figure is higher.**

WHY THE CROSS-CHECK MATTERS

We worked out the Rp52.7M figure once before, from a single source. A separate pull of the line items today puts **CPU and memory alone at Rp40.93M over 30 days**. That is consistent with the earlier figure, and it **rules out being wrong by a factor of ten**. The workflow database is 11.7% of the production project's Cloud SQL bill — a big database, but one of many.

4.5%

THE WORKFLOW TIER'S SHARE OF ITS OWN PRODUCTION PROJECT. IT IS 3.1% OF BRAVO'S TOTAL GCP SPEND.

> **Whatever else is true, the workflow engine is not where Bravo's money goes.**

AGAINST LORA, LIKE FOR LIKE

	BRAVO WORKFLOW TIER	LORA TIER
What is in it	`ms-bpm` pods (Camunda plus all human-task services) and `prod-postgres-bpm-d2bpm`	LPW and LTW workers, gateway, task service, schema service, the Temporal Cloud contract, the ArangoDB licence and its GKE nodes
Monthly, production	≈ Rp 58M	≈ Rp 431M
Applications, August 2026	118,253 by the billing sheet (corrected 2026-09-11) and about 113,000 by the engine's own count — the two now agree to within 5%	171,479 by the billing sheet, or about 135,000 by Temporal's count
Per application	**≈ Rp 490–515**	**≈ Rp 2,500–3,200**
Shape of the cost	Varies with the number of pods; fixed on the database	About 86% fixed — contracts, and capacity we pay for whether we use it or not

The application counts are the weakest part of both columns. The billing sheet's platform totals jump from 109,500 to 237,500 to 247,900 in two months, and nothing else supports that jump. The likeliest explanation is that an application started in LORA is *counted a second time* in Bravo when it books at go-live. If that is right, Bravo's count is too high and its real cost per application is above Rp490–515 — though not enough to close a gap of 5 to 6.5 times. **Corrected 2026-09-11:** the sheet's August Bravo count was in fact too *low*, a partial month; at 118,253 it now agrees with the engine meter to within 5%.

Cost per application, and which way it is moving

	JUN 2026	JUL 2026	AUG 2026	DIRECTION
Bravo applications	106,722	117,996	**118,253**	+0.2% — flat
Bravo support tickets (stable categories)	705	644	751	+82% since January
Bravo stuck-application rate	0.364%	0.348%	**0.452%**	worsening
Workflow tier cost	Roughly flat — one deployment, one database			flat

COST PER APPLICATION IS RISING, AND ENGINEERING IS NOT THE CAUSE

Bravo's platform cost is basically **fixed**, and its volume falls as the migration proceeds — **though August was flat**. So every application we migrate away pushes the cost per application up. The support cost per application is rising faster still: the stuck-application rate rose about **30%** between July and August (0.348% to 0.452%), on flat volume.

HELD LOOSELY: BRAVO KEEPS THE HARD CASES

A platform that is being drained keeps whatever gets migrated last — the awkward products, branches and edge cases. That is a real alternative explanation, and **we have not controlled for it.** It also fits a second explanation: a fixed-flowchart design under constant product change.

RP58M IS NOT A FLOOR TO PLAN AGAINST

At **40,000 applications a month**, that same Rp58M works out at **about Rp1,450 each**. The gap to LORA *halves*, with neither team doing anything.

LORA is moving the other way, and that is the fact the migration case rests on.

About 86% of LORA's cost is fixed. Between June and August it took **23% more volume, as measured by Temporal, for 3% more spend**. Each extra application costs it almost nothing. Bravo's cost per extra application is not zero — it is pods and a database — but it is small.

The cost drivers, ranked. Three of them are defects.

#	DRIVER	MONTHLY	WHAT IT IS
1	Cloud SQL, whole production project	Rp 450.7M	Real work. The workflow database is Rp52.7M of it.
2	Compute Engine, whole production project	Rp 394.4M	Real work
3	Cloud Logging	**Rp 140.5M**	**Mostly waste**
4	Cloud Storage	Rp 86.5M	Real work
5	Test and staging (`bravo-project-nonprod`)	**Rp 573.5M**	**31% of all Bravo spend. A governance problem.**
6	Google Maps platform	**Rp 64.0M**	**Real work, but see the geolocation defect**
7	Redis (Memorystore)	Rp 51.0M	Real work. This is also the cache keyed on NIK that was flagged for personal data.
8	Vertex AI, Vision and Gemini	Rp 56.4M	Real work

THE FLOOD OF 404S COSTS MONEY TOO

One endpoint, `/bpm/v1/partnership-configuration/feature-configuration`, returns 404 **266,767 times a week**. The handler behind it, `PartnershipConfigurationController.getByApplicationId`, is the **busiest handler in the whole service** — 73,056 calls in 48 hours. Each one is a web request, a JPA `repository.operation` query against `prod-postgres-bpm-d2bpm`, and at least one log line. It is a small slice of a large database bill. But it is money spent *to produce an error*, on two-thirds of surveyor sessions.

Cloud Logging costs Rp140.5M — 2.4 times the workflow tier

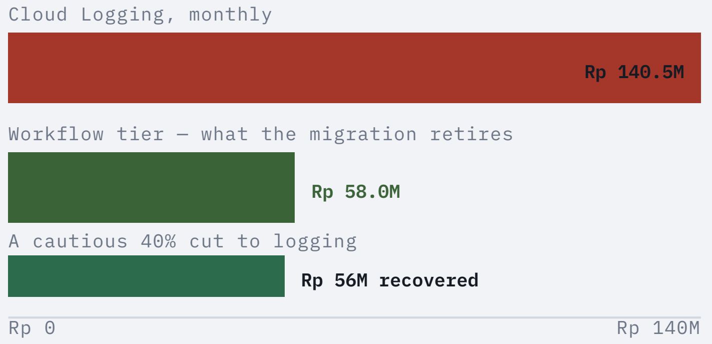
`prod-ms-bpm` alone writes **1,116,135 log lines a week**. **525,181 of them, or 47%, are errors.** Two settings explain most of that, and both are visible in the code.

ONE SETTING TURNS ON FULL REQUEST LOGGING FOR EVERY CLIENT

`loggerLevel: full` is set globally on Feign. So **all 113 HTTP clients log the complete request and response body**. That means PEFINDO, SLIK, Dukcapil and CONFINS data goes into Cloud Logging as JSON. *It is a cost problem and a data-protection problem at the same time.*

STACK TRACES ARE LOGGED ONE FRAME PER LINE

The most common error "patterns" in the log stream are literally `at org.springframework.…` (874 a week) and `at java.base/…`. **One exception becomes dozens of lines we pay for.** A single Camunda job failure writes *two* lines at *two* different severities. On top of that, **38,198 `ENGINE-09004` warnings a week** come from pods re-reading all 53 workflow model files every time they start.



This is the biggest line in Bravo's bill that we can actually do something about. It is larger than the thing the migration would retire. And the fix is a configuration change.

Even a cautious 40% cut saves about Rp56M a month. **That is the entire workflow tier, recovered without migrating anything.**

Rp64M on Maps while location capture fails — and 31% of spend nobody has opened

DEFECT 2 · RP64.0M ON MAPS, AND 20,690 LOCATION FAILURES A WEEK

The Geocoding, Places and Maps APIs are **costs from the surveyor app**: a field surveyor's device looking up addresses and capturing its position. At the same time, browser monitoring on the Surveyor Platform records `Error getting location: "[GeolocationPositionError]"` **10,365 times** and `Unable to get current position` **10,325 times** in seven days.

WE ARE NOT CLAIMING ONE CAUSES THE OTHER

Nobody has put these two facts side by side before, and they sit at *different layers*: one is a browser failure, the other is a server-side API call. But **we spend Rp64M a month on location services, on a console where location capture fails about 20,700 times a week.** Nobody has checked whether the failed attempts are also *billed* attempts. That is one day of work with a real number attached to it.

DEFECT 3 · TEST AND STAGING ARE 31% OF BRAVO'S SPEND

0.44

TEST AND STAGING FOR EVERY RUPIAH OF PRODUCTION — RP573.5M AGAINST RP1,300.3M

For comparison: roughly half of LORA's task-service and ArangoDB bills are SIT and UAT too. But LORA's test environments sit inside a much smaller total.

NEVER EXAMINED

Nothing in this analysis tells us what runs in `bravo-project-nonprod`, or whether it should. At this size it is the *second-biggest saving available*, and nobody has opened it.

What each saving is worth

SAVING	WORTH PER MONTH	AVAILABLE	CONFIDENCE
Cut Cloud Logging. Turn off global request-body logging, write each exception as one event, and stop the model re-parsing warnings.	Rp 50–90M	now	high — it is only configuration
Review `bravo-project-nonprod`	unknown, up to **Rp 573.5M**	now	low — nobody has looked
Check the Maps spend against the failed location calls	up to **Rp 64.0M**	now	low — it needs a day of work first
Keep Camunda history for less than 90 days, or stop recording it at `full`	part of Rp 52.7M	now, with a data-retention decision	medium — after `application_status_log`, this is the audit trail
Fix the `feature-configuration` 404	small	now	high — it removes 266,767 wasted database round trips a week
Retire Bravo's workflow tier (the migration)	**Rp 58M in production, Rp 70–100M with test and staging**	**when the migration finishes**	high
For scale: the entire LORA Temporal Actions programme	*Rp 7.2M*	*now*	*high*

Two conclusions follow, and they are uncomfortable together.

The migration is still the biggest structural saving. It is not the biggest saving available this quarter.

Cloud Logging is bigger, it is available now, and it depends on nothing else. **A team that fixed only the logging settings would save more in September than retiring Bravo's whole workflow tier will save when the migration finishes.**

AND RETIRING BRAVO DOES NOT MAKE LORA'S TIER CHEAPER

About 86% of LORA's Rp431M is fixed, and three of its four savings are **locked in time**. Roughly 31% of the node cost sits on a three-year commitment for N2 CPUs. The Temporal contract can be renegotiated at **about March 2027**. And the five idle worker versions cannot be retired while about half of LORA's loans never reach a finished state — a reliability problem showing up as a cost. **The cost case for the migration is one platform instead of two, at almost no cost per extra application. It is not that the cheaper platform wins.**

Eight actions

1 Cut the Cloud Logging bill · S, DO THIS FIRST — RP50–90M A MONTH

Turn off `loggerLevel: full` globally, then switch it back on for the few clients that need it. That also stops request bodies containing customer data from reaching Cloud Logging. Write each exception as one structured event instead of one line per stack frame. Fix the workflow models so `ENGINE-09004` stops recurring.

This is the largest saving in the document, the fastest, and the lowest risk.

2 Open `bravo-project-nonprod` · S TO LOOK — UP TO RP573.5M A MONTH

This pack has never examined 31% of Bravo's spend. Produce a breakdown by service, and a list of what runs there and why.

Even a 20% cut is worth more than the migration.

3 Find out whether failed location calls are billed · S — UP TO RP64.0M A MONTH

We spend Rp64M a month on Maps APIs, on a console that fails to get a location 20,690 times a week. Find out whether the two are connected before deciding anything else.

We identified the owner on 2026-09-10.

The calls are made from `bravo-surveyor-console`, which uses `@react-google-maps/api` and runs to about 307,000 lines of code. It belongs to squad

`LN` — Team Surveyor & Verificator

, which already has 286 tests gating its pull requests, so there is somewhere to add a regression test once this is fixed.

4 Decide the Camunda history question explicitly · S–M

Recording history at `full` and keeping it for 90 days on Cloud SQL costs real money. After day 91, the only record of what happened to a loan is `application_status_log`, written by a database trigger. Either keep history for less time and accept that, or keep it as it is and stop calling it a cost problem. Either way,

make it a decision instead of a default.

5 Fix the `feature-configuration` 404 · S

266,767 wasted round trips a week, through the busiest handler in the service. The money is small, the load is real, and it is a live defect either way.

6 Get the billing owner to define the application counts · DAYS

This is the largest source of error in the whole pack.

In every per-application figure here, the top number is verified and the bottom number is disputed. We need definitions for the `Bravo total app` and `Lora total app` columns — above all, whether an application that started in LORA is counted a second time in Bravo when it books at go-live.

7 Publish the workflow tier as a standing line · S

The Rp58M figure is the one that matters for the migration decision, and today someone has to rebuild it by hand from line items every time. A saved FinOps view would let us track the migration's value month by month.

8 Do not quote Rp1.6B as the migration saving · S, DOCUMENTATION

Bravo's whole estate costs Rp1.887B, and most of that is the shared data services LORA depends on. Those do not retire. The real figure is

Rp58M in production, or Rp70–100M including test and staging

. We withdrew this claim once already. It should not come back.

WHAT TO DO / PLAN

30, 60, 90 days

Eight recommendations, in phases. Most of this needs **FinOps and Platform time, not Squad S&U engineers.** That matters, because the biggest saving here is a configuration change, not the migration.

THE RULE FOR ORDERING THIS WORK

Savings that are configuration come before savings that are migration. Cloud Logging costs **Rp140.5M a month — 2.4 times the whole Rp58M tier the migration would retire**, and it is a configuration change available now. It starts in week one. **The migration saving is not in this plan at all.** It arrives when the migration finishes, and nothing here makes that sooner.

SHARED WITH OTHER DOCUMENTS

Recommendation 5 is also rec 1 in **bravo-observability.md** and rec 1 in **bravo-delivery.md**. Recommendation 3 goes with rec 2 in **bravo-delivery.md**. Phased the same way in each, so do them once.

Days 0–30

START THE LARGEST SAVING, FIX THE COUNTS

- 1 — **Turn off `loggerLevel: full` globally**, then switch it back on only where it is needed. *S&U and Platform · 3 days.* **The largest single saving in the pack.** Upstream request bodies stop entering Cloud Logging, closing a cost problem and a data-protection problem together. The drop should show on the September bill.
- 6 — **Define `Bravo total app` and `Lora total app`** — above all, whether an application started in LORA is counted again in Bravo at go-live. *FinOps and the billing owner · 2 days.*
- 5 — **Fix the `feature-configuration` 404.** *S&U · 2 days.* The investigation sits in the observability plan.
- 7 — **Publish the workflow tier as a standing FinOps view.** *FinOps · 1 day.* The Rp58M figure becomes a chart we track.
- 2 — **First look at `bravo-project-nonprod`** — a breakdown by service of Rp573.5M a month. *FinOps and Platform · 2 days.*
- **Correct the Rp1.6B figure** wherever it survives

Days 31–60

FINISH THE CUT, ANSWER THE OPEN QUESTIONS

- 1 — **Write each exception as one structured event**, not one line per stack frame. Fix the workflow models so `ENGINE-09004` stops recurring. *S&U · 3 days.* Completes the **Rp50–90M a month** saving. The error share falls from 47%, and the log stream becomes something you can count.
- 2 — **A plan to cut `bravo-project-nonprod`.** *FinOps and Platform · 3 days.* Either a plan to shut things down, or a written reason for Rp573.5M a month.
- 3 — **Check the Maps spend against the failed location calls.** *FinOps and S&U · 2 days.* Either we fix a defect and money comes back, or the spend is justified and stops being an open question.
- 4 — **Decide the Camunda history question.** *S&U and Compliance · 2 days.* A decision on the record, instead of a default.

Days 61–90

VERIFY THE SAVINGS LANDED

- ✓ — No new cost work. **This phase is verification, because a saving you cannot see on a later invoice is a plan, not a saving.**
- 1 — September and October **Cloud Logging** against the August baseline of Rp140.5M. Target: a cut of Rp50–90M a month.
- 7 — The **workflow tier's standing view** across three months. Confirm the Rp58M figure, and see which way it moves as volume drains.
- 6 — **Recompute the cost per application** on the agreed counts — the first figure in the pack that is not provisional.

Deferred

WITH TRIGGERS

- 4 — **Carrying out the Camunda history change.** A data-retention change needing compliance sign-off, not an engineering task. *Trigger: the decision in phase 2.*
- 2 — **Acting on `bravo-project-nonprod`.** Phase 2 produces the plan; what we do next depends on what it finds. *Trigger: the phase-2 plan.*

What changes when this is done

RECOMMENDATION	EXAMPLE WHEN DONE
Logging configuration fixed	We recover Rp50–90M a month, which is more than the migration saves. And PEFINDO, SLIK, Dukcapil and CONFINS request bodies stop landing in Cloud Logging.
Test and staging examined	The second-largest number in Bravo’s bill has an owner and a reason, or a plan to shut it down.
Maps spend explained	Either we fix a defect and tens of millions of rupiah come back, or the spend is justified and stops being an open question.
History decision made	“What happened to this loan on day 120” has a documented answer instead of an accidental one.
Application counts defined	Every rate in this pack stops being provisional: cost per application, intervention rate, and completion without intervention.
Workflow tier tracked	The migration’s financial value is a chart, not an argument.
Rp1.6B taken out of circulation	Nobody plans against a saving that was withdrawn in September 2026.

> **The cheapest thing on this list is worth more than the migration, and it is a configuration change.**